# Project Vela — Master Cookbook

v0.6.0 • beginner-friendly • multi-AI workflow • offline Windows 11 media player

**What this is:** the complete “how to build + how to automate” manual for Project Vela.
**What’s inside:** requirements (from your chat), architecture, repo map, step-by-step workflows for Cursor/Jules/ChatGPT/Gemini/GitHub, and an extensive prompt cookbook.

**Important:** licensing notes are an engineering checklist, not legal advice.

**Source types used:** (1) your requirements, (2) this repo’s code/docs, (3) official developer documentation, (4) common industry practice.

## Contents


1 1. We are building (anti-drift summary)
2 2. Non-negotiables + defaults/toggles
3 3. Chosen stack + licensing reality
4 4. Repo map (where things live)
5 5. Your error-proof workflow (exact order)
6 6. Tool setup (Cursor, Jules, ChatGPT, Gemini, GitHub)
7 7. Conflict prevention (Cursor vs Jules) protocol
8 8. Automation: CI + packaging + release
9 9. Prompt cookbook (copy/paste library + summaries)
10 10. Troubleshooting + what to do when AI goes weird
11 11. Open questions (things to decide soon)
12 12. Sources (clickable links)

# 1) What we are building (anti-drift summary)

**Project Vela** is a **Windows 11** desktop app for **offline** audio/video playback at conferences/live events, designed for an operator controlling playback during a show.

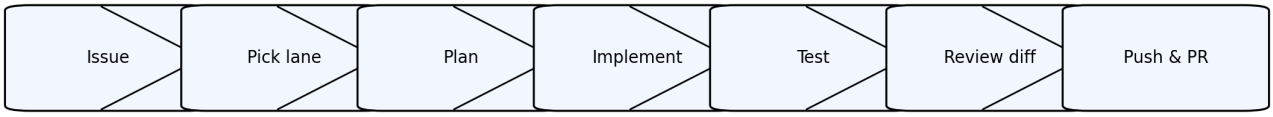
**Source type:** User requirements (this chat) + docs/SPEC.md (repo).

## Core operator story

- Drag files in to build a playlist.
- Watch a folder (including subfolders) to auto-add new media.
- Play reliably. If something fails: output goes black, operator sees error, logs are saved.
- Hit PANIC anytime: output black plus audio mute (default).
- Cue next item in Preview window (paused on first frame, muted by default).

**Source type:** User requirements + docs/SPEC.md (repo).

**Project Vela — Golden Loop (One Issue → One PR)**




# 2) Non-negotiables + defaults/toggles

**Rule:** if any AI suggests behavior that violates these, you reject the change.

| Area | Requirement | Default / toggle |
|---|---|---|
| Offline | Playback must work without network. | Always on (no toggle) |
| Stability | Never crash. On playback error: output black + operator prompt + logs. | Always on (no toggle) |
| Hotkeys | Hotkeys work only when app is focused. | Always on (no toggle) |
| Panic blackout | Hotkey + UI button; output black; audio mute default ON; leaving panic stays paused by default. | Settings toggles for audio mute + auto-resume |
| Playlist end | Stop at end by default; optional loop back to top. | Loop toggle in Settings |

| | | |
|---|---|---|
| Preview window | Cue next item paused on first frame; preview muted by default. | Preview audio monitoring toggle |
| Folder watch | Watch local folder + subfolders; auto-add new files to playlist only. | Watch enabled toggle + include subfolders toggle |
| Packaging | x64 only; per-user installer; no admin required. | Always on (no toggle) |

**Source type:** docs/SPEC.md (repo) + your chat answers (user requirements).

# 3) Chosen stack + licensing reality

Chosen starter stack (your Option B): **.NET + Avalonia + LibVLCSharp + Velopack**.

**Source type:** docs/DECISIONS.md (repo) + dependency metadata (NuGet).

## Why this stack (in plain English)

- **Avalonia** gives you a modern Windows-like UI and can ship as a desktop app.
- **LibVLCSharp** wraps VLC's media engine so you get broad codec/format support (best-effort).
- **Velopack** helps produce a downloadable installer plus update packaging.

**Source types:** Repo decisions + official docs + package metadata.

## Licensing notes (not legal advice)

LibVLC/LibVLCSharp are LGPL-licensed. Selling software can be compatible with LGPL, but you must comply with license obligations. This starter includes a licensing checklist in docs/LICENSING.md and THIRD_PARTY_NOTICES.md.

**Source types:** GNU LGPL license text + NuGet package license metadata + repo docs.

## 4) Repo map (where things live)

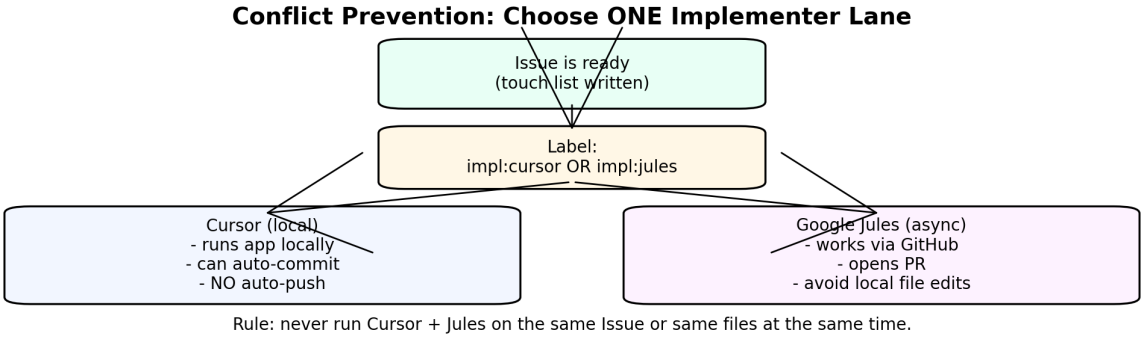
If you remember nothing else: **docs/SPEC.md is the single source of truth**.

| Path | What it is | When you touch it |
|---|---|---|
| docs/SPEC.md | Product requirements & defaults/toggles | Before coding; when requirements change |
| docs/WORKFLOW_AI.md | How Cursor/Jules/ChatGPT/Gemini cooperate | When workflow drifts |
| docs/PROMPTS.md | Copy/paste prompts for all AIs | Every new Issue |
| src/ConferencePlayer.App | Avalonia UI + ViewModels | UI changes and operator workflows |
| src/ConferencePlayer.Playback | IPlaybackEngine + LibVLCSharp implementation | Playback behavior |
| src/ConferencePlayer.Core | Settings, playlist model, folder watch, logging | Stability + data model |
| .github/workflows | CI + release automation | When builds/releases fail |
| .cursor/rules | Cursor guardrails | When Cursor starts hallucinating |
| AGENTS.md | Instructions that Jules reads | When Jules needs better guidance |

**Source type:** Repo artifact (file layout).

# 5) Your error-proof workflow (exact order)

This is the “do it the same way every time” loop. The goal is: automation without chaos.




## A. Write the Issue (GitHub)

- Goal / Non-goals / Defaults & toggles / Acceptance criteria
- Touch list (files allowed to change)
- Pick one implementer lane label

## B. Plan (ChatGPT)

- Ask for plan + risk list + verification steps
- Ask it to quote docs/SPEC.md headings (prevents drift)

## C. Red-team (Gemini)

- Ask Gemini to attack the plan: crash paths, monitor changes, codec edge cases
- Add missing acceptance checks back into the Issue

## D. Implement (Cursor OR Jules)

- Cursor: local iteration; run tests; commit; stop; no push
- Jules: run task from Issue; review plan; let it open PR

## E. Verify (You)

- Smoke test (docs/SMOKE_TEST.md)
- Glance at diff vs Issue + touch list
- Push (Cursor lane) and open PR

## F. Merge + release

- Merge only when CI is green
- Use Velopack workflow to build installer artifacts

**Source types:** Common industry practice + official docs + your chosen workflow preference.

# 6) Tool setup (beginner friendly)

## 6.1 GitHub setup

- Enable protected branch rules for **main**: require PR, require status checks (CI).
- Use Issues as the source of truth: every change starts with an Issue.
- Use labels: impl:cursor OR impl:jules (never both).

Link: GitHub protected branches docs.

**Source type:** GitHub official documentation + repo conventions.

## 6.2 Cursor setup (local agent)

- Open the repo in Cursor and confirm **.cursor/rules** exists.
- Enable agent terminal only if you will review executed commands.
- Let Cursor commit to your branch; you push after reviewing the diff.

Links: Cursor rules • Cursor agent terminal • Cursor security.

**Source types:** Cursor official documentation + your workflow preference.

## 6.3 Google Jules setup (async PR agent)

- Jules reads **AGENTS.md** at repo root for tool conventions.
- Start a task from a GitHub Issue by applying the label **jules**.
- Jules runs tasks in a VM that clones your repo and installs dependencies.

Links: Jules docs • Running tasks.

**Source type:** Jules official documentation.

## 6.4 ChatGPT setup (planner + reviewer)

- Create a ChatGPT **Project** named “Project Vela”.
- Upload repo docs so ChatGPT can quote them (SPEC, WORKFLOW, PROMPTS).
- Put the Universal Header into your Project instructions.

Links: Projects in ChatGPT • File uploads FAQ.

**Source type:** OpenAI official documentation.

## 6.5 Gemini Ultra setup (red-team)

- Use Gemini for red-team thinking: how could this crash or fail in production?
- Ask for offline experiments: CPU usage, long-play stability, monitor disconnect.

Link: Gemini thinking documentation.

**Source types:** Google official documentation + common industry practice.

# 7) Conflict prevention protocol (Cursor vs Jules)

**Goal:** you should almost never see a merge conflict. Conflicts are usually a workflow issue, not a technical requirement.

## The rule set

- **One implementer per Issue:** impl:cursor OR impl:jules.
- **Touch list required** before implementation starts.
- **File ownership:** the agent who owns the Issue is the only one editing those files.
- **Cursor commits, human pushes:** reduces accidental remote pollution.
- **If you must switch lanes:** close/merge the current PR first, then relabel the Issue and update touch list.

**Source types:** Common industry practice + your chosen workflow preference.

## Emergency procedure: I accidentally ran both AIs

- Stop one agent immediately.
- Pick which branch/PR to keep.
- Cherry-pick only the necessary commits onto a clean branch.
- Update the Issue with the new touch list and lane label.

**Source type:** Common industry practice (git conflict recovery).

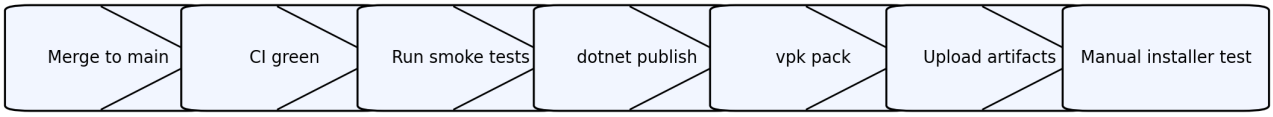
# 8) Automation (CI + packaging + release)

Automation makes you faster, but only if it's predictable. This repo ships with:

- **CI**: dotnet restore/build/test on PRs and pushes.
- **Release workflow**: packages a Velopack release artifact on demand.
- **Lane guard**: warns if both impl:cursor and impl:jules labels are set.

**Source type:** Repo artifact (.github/workflows).

## Release pipeline




## Velopack notes (per-user, no privileged directories)

Velopack's Windows packaging documentation describes installation under the user profile (not system folders) and includes a section on preserving user files and settings across updates.

Links: Windows packaging • Preserving files.

**Source type:** Velopack official documentation.

# 9) Prompt cookbook (copy/paste library + summaries)

These prompts are designed to be beginner-safe: they force plans, touch lists, tests, and admissions of uncertainty.

**Critical rule:** if you do not paste the Issue + acceptance criteria + touch list, the AI will guess. Always include them.

## Prompt index (summary for each)

| Tool | Prompt name | When to use | Expected output |
|---|---|---|---|
| ChatGPT | Issue drafting | Before coding | Issue text + acceptance criteria |
| ChatGPT | Lane decision | Start of every Issue | impl:cursor vs impl:jules + touch list |
| ChatGPT | Mini design note | Playback/monitor/file-watch work | Approach + failure modes |
| ChatGPT | Beginner test plan | Before implementation | Unit + smoke checklist |
| ChatGPT | PR review | Before merge | Blockers + missing tests |
| Cursor | Implement Issue (commit/no push) | impl:cursor Issues | Code + tests + local commit |
| Cursor | Bug fix from logs | When you have a log snippet | Root cause + smallest fix |
| Cursor | Add setting/toggle | Any new option | UI + persistence + docs update |
| Cursor | Add hotkey | New operator control | Focused-only hotkey + smoke test |
| Jules | Implement Issue (small PR) | impl:jules Issues | Async PR + verification steps |
| Jules | Docs only | Safe background cleanup | Docs PR only |
| Jules | Tests only | Increase coverage | Tests PR only |
| Gemini | Red-team plan | After planning | Failure modes + mitigations |
| Gemini | Break the release checklist | Before shipping | 20 stress tests + expected behavior |

**Source type:** Common industry practice + this repo's workflow docs.

## Copy/paste prompts

### ChatGPT — Lane decision + touch list

```
You are working on Project Vela.

Inputs:
- GitHub Issue: <PASTE>
- Relevant spec excerpt (docs/SPEC.md): <PASTE>

Task:
1) Choose ONE lane: impl:cursor OR impl:jules (with reasoning).
2) Produce a touch list (files allowed to change).
3) Produce a 6-10 step plan.
4) Produce a verification plan (unit + smoke).
5) If anything is unknown, say: "I am uncertain about this" and ask for the missing info.
```

### Cursor — Implement Issue (commit allowed, no push)

```
Implement Issue #<N> locally.

Constraints:
- Respect docs/SPEC.md and docs/ARCHITECTURE.md.
- No new dependencies unless explicitly allowed.
- Keep changes small.
- You may commit, but DO NOT push unless I ask.

Output:
- Summary
- Files changed
- Commands run
- Smoke test steps
```

### Cursor — Bug fix from logs (smallest fix first)

```
Fix a bug using this information.

Repro steps:
<PASTE>

Log snippet:
<PASTE>

Requirements:
- Never crash posture: output black + operator prompt + logs.
- Add or improve logging so future failures are diagnosable.
- Keep the fix minimal.

Output:
- Root cause hypothesis (file + method)
- Smallest safe fix
```

```
- Tests to add/update
```

## Jules — Implement Issue (small PR)

```
Implement GitHub Issue #<N>.

Constraints:
- Respect the Issue touch list.
- Keep PR small (~300 LOC).
- No new dependencies unless explicitly permitted.
- Update docs/SMOKE_TEST.md if behavior changes.
- Add/update tests where possible.

Deliverable:
- PR with a clear summary + verification steps.
```

## Gemini — Red-team the plan

```
Act as a red-team reviewer for Project Vela.

Given this plan:
<PASTE>

Attack it for:
- crash risks
- file watch pitfalls
- multi-monitor pitfalls
- operator workflow hazards

Output:
- Failure modes
- Mitigations
- Missing acceptance checks
```

## Gemini — Break the release checklist

```
Act as QA lead for an offline Windows media player.

Given features:
- panic blackout
- preview cue next
- folder watch (subfolders)
- playlists + loop toggle

Create 20 short stress tests:
- steps
- expected result
- what to check in logs
```

Full prompt library lives in **docs/PROMPTS.md** (in the zip).

# 10) Troubleshooting + what to do when AI goes weird

Most failures are predictable. Here are the patterns.

### AI changed too many files

- Close PR, revert, rewrite Issue with a tighter touch list.
- Ask ChatGPT to split into smaller Issues.

### AI added a new dependency

- Stop. Ask: do we truly need it?
- If no, revert. If yes, add license review to the Issue.

### Playback error handling missing

- Reject the PR. The app must never crash.
- Add operator-visible error + logs + output black.

### Cursor pushed junk to GitHub

- Revert remote commit, enforce: commit then diff review then push.

### Jules and Cursor both edited the same file

- Pick one PR; cherry-pick clean commits; close the other.
- Re-label Issues and re-assign lanes.

**Source type:** Common industry practice.

## 11) Open questions (things to decide soon)

- Do you want code signing for the installer (recommended for Windows SmartScreen reputation)?
- Do you want an auto-update channel (Velopack can do updates, but you can ship offline-only first)?
- Do you want a standardized set of sample media files for regression testing? (Even 3–5 files helps.)
- Do you want crash reporting beyond local logs? (This can conflict with offline-only unless optional.)

**Source type:** Common industry practice + unresolved product decisions.

## 12) Sources (clickable)

| Item | Link | Source type |
|---|---|---|
| Cursor docs — Rules | https://cursor.com/docs/context/rules | Official developer documentation |
| Cursor docs — Agent terminal | https://cursor.com/docs/agent/terminal | Official developer documentation |
| Cursor security | https://cursor.com/security | Official developer documentation |
| Jules docs — Getting started | https://jules.google/docs/ | Official developer documentation |
| Jules docs — Running tasks (includes Issue label “jules”) | https://jules.google/docs/running-tasks/ | Official developer documentation |
| GitHub docs — Protected branches | https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches | Official developer documentation |
| Velopack docs — Packaging overview | https://docs.velopack.io/packaging/overview | Official developer documentation |
| Velopack docs — Windows packaging | https://docs.velopack.io/packaging/operating-systems/windows | Official developer documentation |
| Velopack docs — Preserving files and settings | https://docs.velopack.io/packaging/preserving-files | Official developer documentation |
| NuGet — LibVLCSharp.Avalonia package (license metadata) | https://www.nuget.org/packages/LibVLCSharp.Avalonia | Official package registry metadata |
| NuGet — VideoLAN.LibVLC.Windows package (bundled native binaries) | https://www.nuget.org/packages/VideoLAN.LibVLC.Windows | Official package registry metadata |
| GNU — LGPL v2.1 license | https://www.gnu.org/licenses/old-licenses/lgpl-2.1.en.html | License text (authoritative) |
| OpenAI Help — Projects in ChatGPT | https://help.openai.com/en/articles/10169521-projects-in-chatgpt | Official developer documentation |

| | | |
|---|---|---|
| OpenAI Help — File uploads FAQ | https://help.openai.com/en/articles/8555545-file-uploads-faq | Official developer documentation |
| Google AI Dev — Gemini thinking | https://ai.google.dev/gemini-api/docs/thinking | Official developer documentation |